La recherche de données

Table des matières

1. Recherche d'un licencié sur son numéro de licence 1
2. Recherche d'un licencié sur son nom à l'aide d'un système d'autocomplétion 2
3. Recherche des licenciés appartenant à une catégorie 3
4. Recherche sur la catégorie, le sexe et le club 4
5. Recherche sur des critères imbriqués 5
    5.1. Les méthodes de la classe Competence (classemetier/competence.php) 6
    5.2. Le script du répertoire ajax 6
    5.3. Le script index.js 6

Guy Verghote

La recherche de données

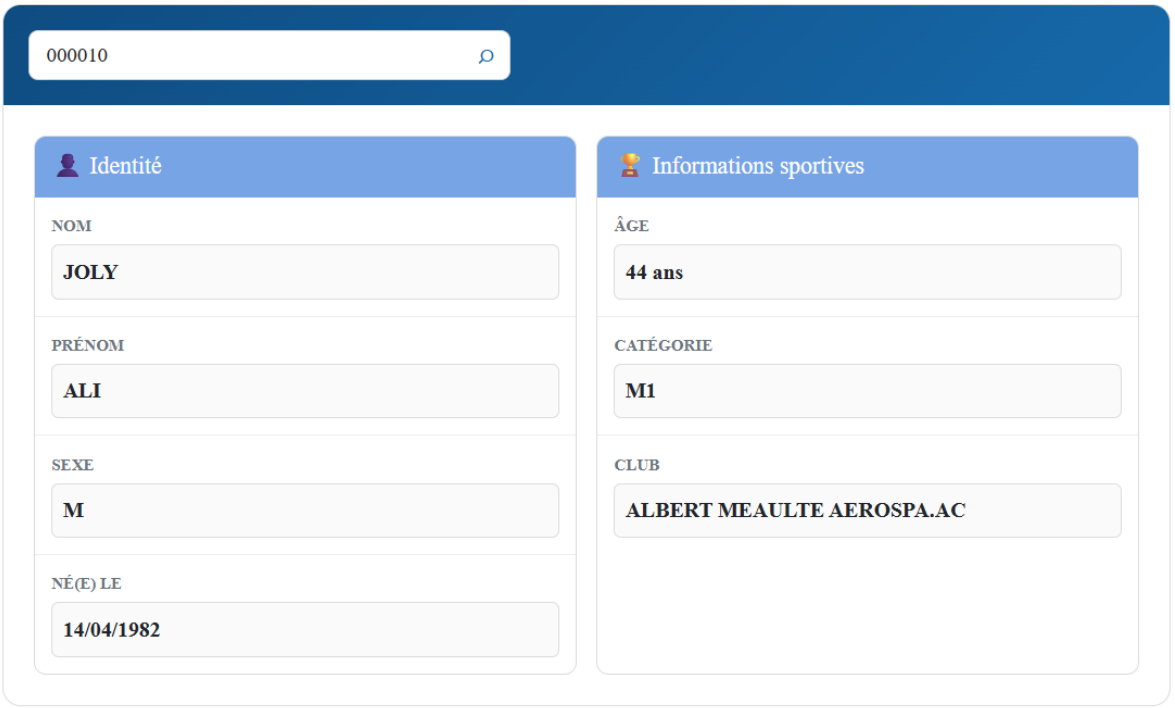
Objectif de l'activité professionnelle

Étudier la mise en place d'une recherche renvoyant un ou plusieurs enregistrements Être capable de reproduire ces mises en place sur d'autres développements

1. Recherche d'un licencié sur son numéro de licence

02 Développement Web> 05 La recherche de données > Recherche un coureur par son numéro de licence

Répertoire public/coureur/getbylicence



L'âge est une donnée calculée à l'aide de la fonction getAge(date), définie dans le composant fonction/date.js.

La validation du champ de saisie « licence » à l'aide de la touche Entrée déclenche un appel AJAX vers le script ajax/getbylicence.php. Le numéro de licence est transmis à ce script via une requête POST.

En cas de succès, la réponse contient les coordonnées du coureur au format JSON. En cas d'erreur, elle contient le détail de l'erreur rencontrée, également au format JSON.

Client : index.js		Serveur ajax/getbylicence.php
appelAjax({ url: "ajax/getbylicence.php", data: {licence: licence.value}, ….	→ {'licence' : '000010'} ← {'dateNaissanceFr' :"14/04/1982" 'idCategorie':"M1" 'licence' :"000010" 'nom' : "Joly" … 'sexe : "M" }	Requete::*exigerPost*(); $licence = Requete::*postString*('licence'); Vérification … Envoi de la réponse dans le format JSON en utilisant une méthode de la classe ReponseJson

02 Développement Web> 01 Présentation > 07 Tester une requête POST protégée avec Insomnia

1. Réaliser sous Insomina les tests fonctionnels du script php appelé par la fonction appelAjax()

2. Expliquez en détail le fonctionnement du code JavaScript ci-dessous, puis réécrivez-le en utilisant une fonction fléchée (notation =>). Précisez notamment le rôle de this dans la version originale et expliquez la modification nécessaire dans la version avec fonction fléchée.

```
licence.onchange = function() {
    if (this.checkValidity()) {
        rechercher(this.value);
    } else {
        afficherSousLeChamp('licence');
    }
};
```

2. Recherche d'un licencié sur son nom à l'aide d'un système d'autocomplétion

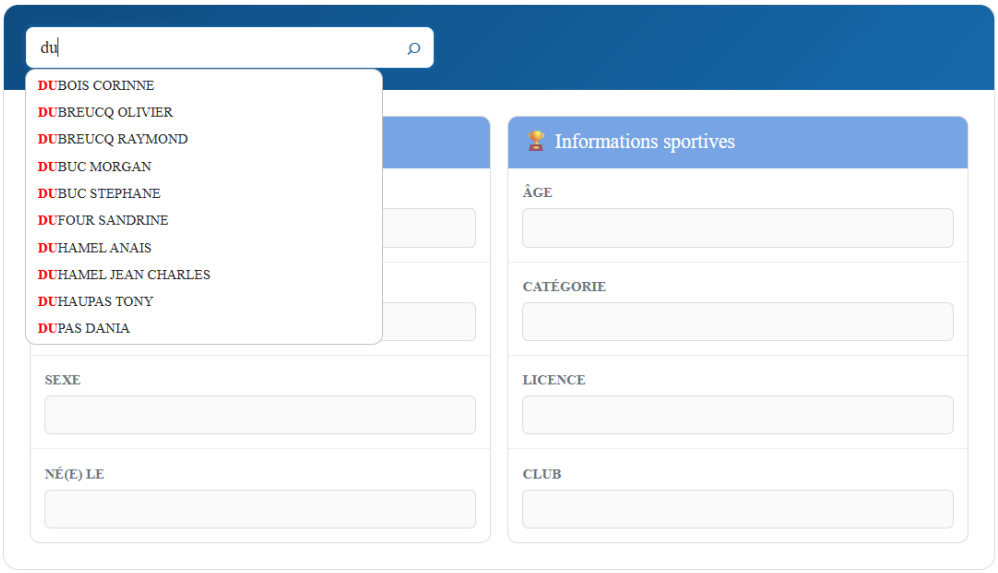
Répertoire public/coureur/getbyname et public/coureur/getbyname2

Il est difficile de connaitre le numéro de licence de tous les coureurs, en revanche on connait plus facilement leur nom et prénom. Proposer de sélectionner un coureur à partir d'une zone de liste n'est pas un bon choix car le nombre de coureurs est élevé.

En revanche, en mettant un place un système d'autocomplétion, il est possible de sélectionner un coureur sur son nom en saisissant simplement le début de son nom.

Nous allons utiliser le composant **autocomplete.js** (https://tarekraafat.github.io/autoComplete.js) pour mettre en place le système d'autocomplétion.

Ce composant est chargé depuis le fichier index.html via l'url de son CDN
Seul le script JavaScript est chargé, car le feuille de style associé n'est pas compatible avec les styles appliqués sur nos champs de recherche



Deux versions de cette interface sont présentées

02 Développement Web> 03 Élément pratiques > Composant client > autocomplete.js

3. Enumérez les différences entre les deux versions
4. Faire ressortir les avantages et inconvénients des deux versions.
5. À partir de l'URL utilisée comme source par le composant Autocomplete, réaliser avec Insomnia les tests fonctionnels permettant de vérifier le comportement de cette Url.
6. Quelle est la meilleure solution dans ce contexte ?

3. Recherche des licenciés appartenant à une catégorie

Répertoire coureur/getbycategorie



Nous souhaitons récupérer tous les licenciés appartenant à une catégorie. Une zone de liste est proposée afin de sélectionner la catégorie.

7. Quelles sont les données transmises par le contrôleur au client ?
8. Expliquer pourquoi certaines catégories sont grisées et comment cela a pu être mise en place.
9. Décoder l'expression régulière servant à contrôler le format de la catégorie côté serveur

Dans le script index.js, nous trouvons l'appel Ajax suivant :

```
function getLesCoureurs(idCategorie) {
    appelAjax({
        url: 'ajax/getlescoureurs.php',
        data: {idCategorie: idCategorie},
        success: afficher
    });
}
```

La propriété success de la fonction appelAjax() utilise ici une fonction de rappel (callback). Il existe plusieurs façons d'écrire cette fonction de rappel.

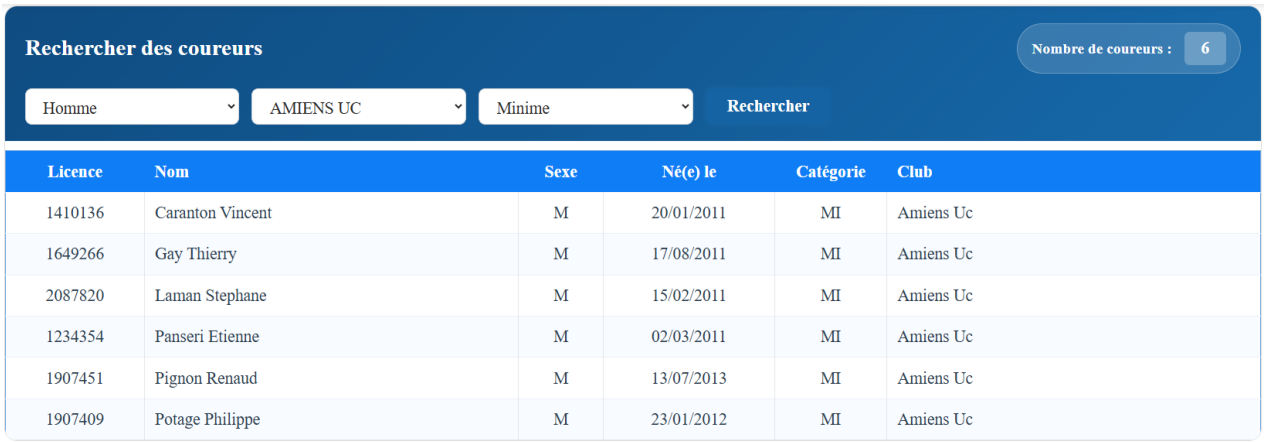
10. Expliquez l'écriture actuellement utilisée : success: afficher. Précisez notamment ce que représente afficher et comment les paramètres sont transmis à cette fonction

11.Proposez une écriture utilisant une fonction anonyme classique.

12. Proposez une écriture utilisant une fonction fléchée. Vous utiliserez la syntaxe la plus concise possible et expliquerez pourquoi les parenthèses autour du paramètre et les accolades peuvent être omises dans ce cas.

4. Recherche sur la catégorie, le sexe et le club

Répertoire coureur/recherchemultiple



Le bouton 'rechercher' permet de lancer la recherche. Il serait possible de se passer de se bouton et de déclencher la recherche sur chaque événement change associé aux zones de liste, cependant le bouton permet de vérifier ici qu'au moins un des critères de recherche est renseigné.



13.Quelles sont les données transmises par le serveur au client lors du chargement de la page ? Indiquer leur origine et expliquer à quoi elles servent dans l'interface.

14.Quelles sont les données transmises par le client au contrôleur situé dans le répertoire Ajax

15.Expliquer comment la méthode Coureur::getBySexeClubCategorie() arrive à générer la requête permettant de répondre à la demande

16.À l'aide d'Insomnia, réaliser les tests fonctionnels du contrôleur ajax/getlescoureurs.php. Vérifier notamment les réponses obtenues pour les différents critères de recherche ainsi que pour les requêtes invalides.

5. Recherche sur des critères imbriqués

Répertoire recherche/competence

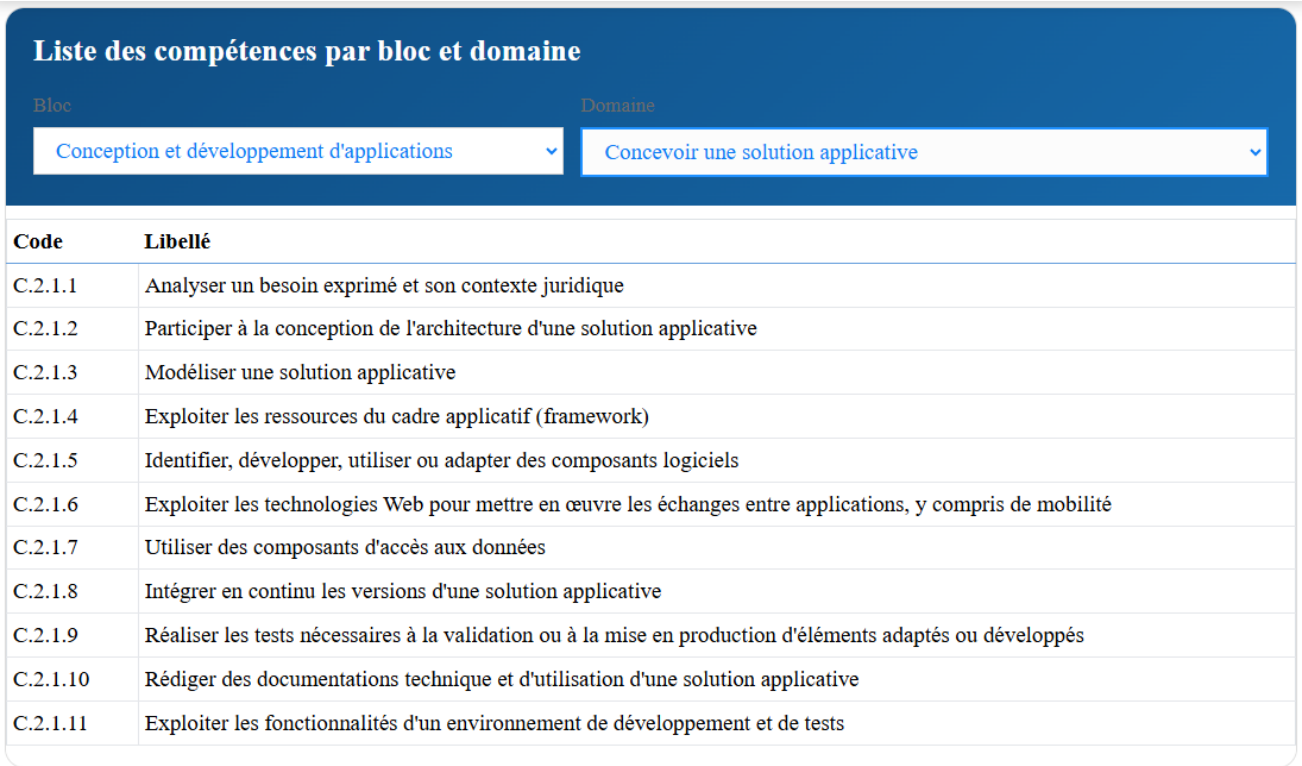
Certains critères de recherche influent sur d'autres critères. On parle alors de critères imbriqués. C'est le cas notamment dans le référentiel SIO où les compétences sont organisées en domaines d'activité qui appartiennent à un bloc du référentiel.

Autrement dit :

À un bloc correspond une liste de domaines.
À un domaine correspond une liste de compétences.

On souhaite afficher les compétences du référentiel pour un bloc, ou un domaine dans un bloc.
La première zone de liste doit afficher les blocs et la seconde doit afficher les domaines correspondant au bloc sélectionné.
Le remplissage de la seconde zone de liste doit donc être dynamique : La sélection d'une valeur dans la première zone modifie les valeurs de la seconde.

L'interface graphique



Code	Libellé
C.2.1.1	Analyser un besoin exprimé et son contexte juridique
C.2.1.2	Participer à la conception de l'architecture d'une solution applicative
C.2.1.3	Modéliser une solution applicative
C.2.1.4	Exploiter les ressources du cadre applicatif (framework)
C.2.1.5	Identifier, développer, utiliser ou adapter des composants logiciels
C.2.1.6	Exploiter les technologies Web pour mettre en œuvre les échanges entre applications, y compris de mobilité
C.2.1.7	Utiliser des composants d'accès aux données
C.2.1.8	Intégrer en continu les versions d'une solution applicative
C.2.1.9	Réaliser les tests nécessaires à la validation ou à la mise en production d'éléments adaptés ou développés
C.2.1.10	Rédiger des documentations technique et d'utilisation d'une solution applicative
C.2.1.11	Exploiter les fonctionnalités d'un environnement de développement et de tests

L'interface positionne deux balises <select id ='idBloc'> et <select id='idDomaine'> sans leur option.

L'entête du tableau est défini en y ajoutant les attributs 'data' permettant la mise en place du tri des colonnes

5.1. Les méthodes de la classe Competence (classemetier/competence.php)

Pour alimenter cette interface nous avons besoin des trois méthodes suivantes de la classe Competence :

getLesBlocs() pour récupérer les blocs

```
select id, libelle from bloc order by libelle;
```

getLesDomaines($idBloc) pour récupérer les domaines associés à un bloc

```
select idDomaine, libelle from domaine where idBloc = :idBloc order by libelle;
```

getLesCompetences($idBloc, $idDomaine = '*') pour récupérer les compétences associées à un bloc ou à un domaine d'un bloc

La condition sur le domaine est facultative ($idDomaine === '*'). Il faut donc générer une partie de la requête dynamiquement en lui ajoutant une clause "and"

17. Observer les méthodes de la classe Competence

5.2. Le script du répertoire ajax

18. Indiquer pour chaque script, le rôle, les paramètres attendus, et tester leur fonctionnement à l'aide d'Insomnia

5.3. Le script index.js

19. Quelles sont les actions réalisées au lancement ?
20. Quel est le rôle de la fonction getLesDomaines et à quel moment est-elle appelée ?
21. Quel est le rôle de la fonction getLesCompetencesDuBloc et à quel moment est-elle appelée ?
22. Quel est le rôle de la fonction getLesCompetencesDuDomaine et à quel moment est-elle appelée ?